allreduce/reduce_bcast

An AgentMPI collective scaling analysis

Abstract

`reduce_bcast` is the composed allreduce: fold everything to rank 0 with a binomial `reduce`, then hand the single result back out with a binomial `bcast`. It costs $2(p-1)$ messages in $2\lceil \log_2 p \rceil$ rounds with fold depth $\lceil \log_2 p \rceil$, and across 21 process counts from 2 to 32 the logged traffic matches that closed form at every size — including all twelve non-power-of-two sizes, because a binomial tree has no remainder path to get wrong. It is AgentMPI's default allreduce, and the family view makes the case for that default stronger than the textbook comparison suggests. Against `recursive_doubling` it uses 2.6× fewer messages, 3.9× less wire volume and 5.2× fewer operator applications at $p = 32$ (31 against 160), and it gives all of that up for nothing in latency: its extra $\lceil \log_2 p \rceil$ rounds are *broadcast* rounds, and a broadcast in AgentMPI forwards a content-addressed handle rather than invoking the operator, so the repository's own cost model puts the two algorithms 0.4% apart in time at every measured size. It also delivers something recursive doubling cannot promise: one value, computed in one canonical order, byte-identical everywhere — at $p = 32$ all 31 broadcast sends carry the same digest, `e29c9c180c62`. The reader's trap is the number zero. The outer `coll.allreduce` record reports `messages_sent: 0` and `tokens_sent: 0` at every size, because a composed algorithm's traffic belongs to its constituents; the analysis tooling now credits it correctly, and an earlier version that did not reported a phantom disagreement at all 21 sizes.

1 What this family is

The `allreduce` collective under the `reduce_bcast` algorithm, measured at 21 process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- **[ok]** All 21 measured sizes logged exactly the number of messages the closed form predicts.
- **[note]** Wall times span 0.015–0.208 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

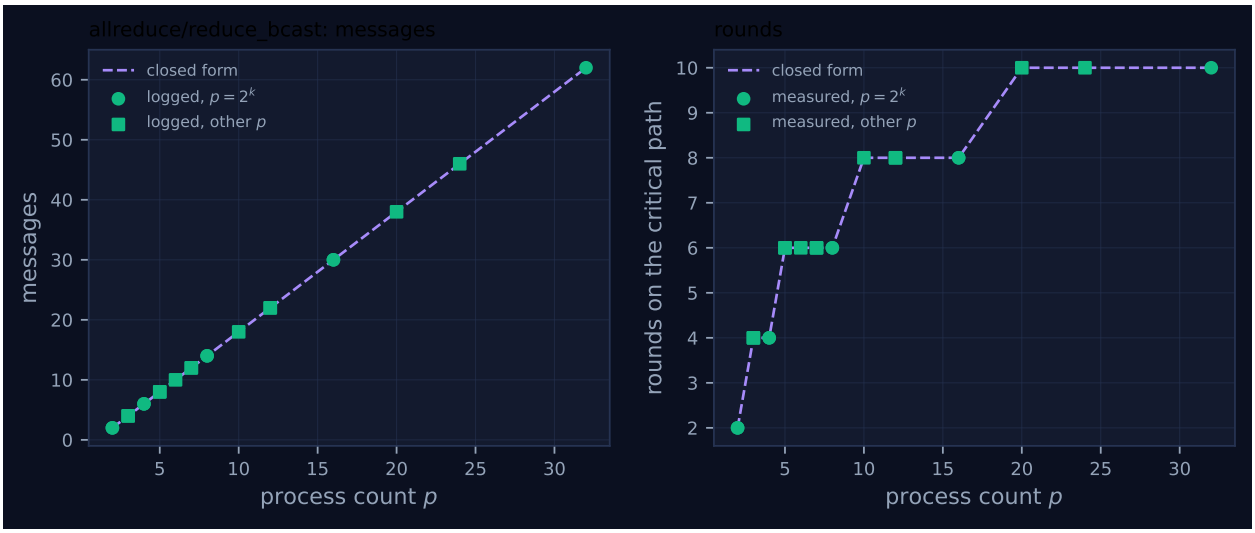

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	2	2	2	2	2	0.015	✓	✓
2	`mb-smoke`	2	2	2	2	2	0.029	✓	✓
3	`mb-free`	4	4	4	4	4	0.030	✓	✓
3	`mb-smoke`	4	4	4	4	4	0.029	✓	✓
4	`mb-free`	6	6	6	4	4	0.032	✓	✓
4	`mb-smoke`	6	6	6	4	4	0.032	✓	✓
5	`mb-free`	8	8	8	6	6	0.056	✓	✓
5	`mb-smoke`	8	8	8	6	6	0.051	✓	✓
6	`mb-free`	10	10	10	6	6	0.062	✓	✓
7	`mb-free`	12	12	12	6	6	0.062	✓	✓
7	`mb-smoke`	12	12	12	6	6	0.048	✓	✓
8	`mb-free`	14	14	14	6	6	0.075	✓	✓
8	`mb-smoke`	14	14	14	6	6	0.073	✓	✓
10	`mb-free`	18	18	18	8	8	0.080	✓	✓
12	`mb-free`	22	22	22	8	8	0.080	✓	✓
12	`mb-smoke`	22	22	22	8	8	0.107	✓	✓
16	`mb-free`	30	30	30	8	8	0.110	✓	✓
16	`mb-smoke`	30	30	30	8	8	0.136	✓	✓
20	`mb-free`	38	38	38	10	10	0.124	✓	✓
24	`mb-free`	46	46	46	10	10	0.165	✓	✓
32	`mb-free`	62	62	62	10	10	0.208	✓	✓

4 How the algorithm scales

4.1 Two trees, and where the rounds come from

`allreduce(algorithm="reduce_bcast")` is eleven lines of `algorithms.py`: call `reduce` to rank 0, call `bcast` from rank 0, and set `rounds = 2 _ilog2_ceil(p)`. Everything about the cost follows from the two constituents.

The binomial `reduce` is the ordinary bottom-up hypercube fold. Rank r's virtual rank is r (the root is 0, so virtual and real coincide), and while its lowest set bit has not been reached it absorbs one child per level and then sends to its parent and stops. Every non-root rank sends exactly once, giving $p-1$ messages; the longest chain from a leaf to the root is $\lceil \log_2 p \rceil$ levels, giving both the round count and the fold depth. The binomial `bcast` is the same tree run downwards: $p-1$ messages, $\lceil \log_2 p \rceil$ rounds, no folds. Summing: $2(p-1)$ messages, $2\lceil \log_2 p \rceil$ rounds, fold depth $\lceil \log_2 p \rceil$. That is exactly the entry in `cost.FORMULAS`, and it is exactly what the table records at all 21 sizes.

The messages panel of Figure 1 is therefore a straight line — the only strictly linear family among the allreduce and allgather algorithms — and the two marker classes are indistinguishable on it, which is the point of this algorithm. A binomial tree is defined by bit tests on the rank, so it degrades gracefully at any p: a rank whose child index exceeds p simply has no child. There is no remainder phase, no renumbering, and no separate code path for the sizes where collective implementations usually go wrong.

The rounds panel is a staircase at twice the height of the usual one, stepping at $p = 3, 5, 9, 17$ to 4, 6, 8 and 10. The steps land where $\lceil \log_2 p \rceil$ steps because both halves step together.

4.2 The composition is internally consistent, and checkable

The claim that the outer round count is the sum of the constituents' is not something to take on faith: each constituent emits its own `coll.*` record with its own `rounds`, and they can be added up. Doing so over the `mb-free` campaign, the sum of the maximum `reduce` round count and the maximum `bcast` round count equals the outer `allreduce` report at every size from 4 to 32: $2+2=4$, $3+3=6$, $4+4=8$, $5+5=10$.

There is one exception, and it is instructive. At $p = 3$ the outer record says 4 rounds while the constituents say $1+2=3$, because both `reduce` and `bcast` default to `flat` rather than `binomial` below $p = 4$ (`algorithm or ("flat" if p <= 3 else "binomial")` in both), and a flat reduce is one round. The outer figure is hard-coded to $2\lceil \log_2 p \rceil$ and does not consult what its constituents actually did, so at $p = 3$ it is one round generous. The closed form agrees with it, being the same expression, so the generated table scores this ✓. It is a two-run edge case with no consequence for anything, but it is the same failure mode as the round discrepancy in the recursive-doubling family: the self-report and the closed form are the same arithmetic, so their agreement is not a check.

4.3 The zero, and why it is not a loss

The outer `coll.allreduce` record reports `messages_sent: 0` and `tokens_sent: 0` at every one of the 21 sizes. Nothing has gone missing. The `_Tracker` for the outer call never has `sent()` invoked on it, because the outer call never sends: it delegates to `reduce` and `bcast`, each of which keeps its own tracker and emits its own record. Counting the outer call's traffic as well would double-count it.

The trace shows the arithmetic directly. In `mb-free__coll-allreduce-reduce_bcast-10` the fabric holds 18 `msg.send` events, and the three collective records say: `reduce/binomial` 9 messages, `bcast/binomial` 9 messages, `allreduce/reduce_bcast` 0. The viewer screenshot displays exactly this — a MESSAGES tile reading 18 above a COLLECTIVES panel whose three rows read 9, 9 and 0 — and the sum is right. `analysis.py` reconstructs the attribution from a hard-coded table, `COMPOSED_ALGORITHMS`, which is taken from the implementations rather than inferred from timing, for a reason worth repeating: the outer record is emitted on completion, *after* its constituents, so temporal containment does not identify the nesting and a timing heuristic would mis-attribute. The `effective_messages` property then adds `nested_messages` to `messages`, which is why the "self-rep." column of the generated table reads 18 rather than 0 and matches the log.

Getting this wrong is not hypothetical. An earlier version of `analyze_family.py` selected the first complete invocation in a run rather than the one matching the family's op, which for a composed algorithm picks up a *constituent*: the whole run's 18 messages were then compared against `reduce`'s closed form of $p - 1 = 9$, and the tool reported a disagreement at all 21 sizes of this family. That was a defect in the analysis tooling and not in the runtime, it is fixed, and it is a useful warning about the shape of the error a composed algorithm invites — a reader who takes the outer zero at face value concludes the collective sent nothing, and a tool that takes it at face value concludes the model is wrong everywhere.

4.4 Fold depth is a per-rank number that only means something at the maximum

The outer record carries `fold_depth`, read from `LAST_STATS` before the broadcast overwrites it — the implementation comments on this explicitly, since the broadcast contributes no folds and would otherwise report zero. What it reads is that rank's *own* fold depth in the reduce tree, not the depth of the value the rank ends up holding. At $p = 10$ the ten `coll.allreduce` records report depths of 4 (rank 0), 2 (rank 4), 1 (ranks 2, 6, 8) and 0 (the five leaves), while all ten ranks finish holding the same fully-folded result, whose depth is 4.

For an exact operator this is bookkeeping. For a lossy one it matters, because $F(d) = (1 - \delta)^d$ applied per rank would say the leaves hold undegraded values, when in fact every rank holds the root's 4-deep fold. The aggregate the viewer shows, MAX FOLD DEPTH 4, is the meaningful figure, and `analysis.py` takes the maximum over ranks for the same reason. A harness reading a single rank's `fold_depth` to estimate the fidelity of what it received would be wrong by up to $\lceil \log_2 p \rceil$ levels.

5 Powers of two and everything else

There is nothing to report here, and that is the result.

The two marker classes in Figure 1 lie on the closed-form line together, at all 21 sizes, on both axes, and the accounting column of the generated table is ✓ at every row. No collective in this family has a remainder path, because a binomial tree does not need one: the tree is generated by `while mask < p` with a `child_v < p` guard, so a non-power-of-two population produces a slightly lopsided tree and nothing else. Rank 0 at $p = 10$ absorbs children 1, 2, 4 and 8 — four levels, of which the last carries only rank 8's two-rank subtree while ranks 2 and 4 carry two-rank and four-rank subtrees. The tree is unbalanced; the message count is not affected, because the count is "every non-root rank sends once" regardless of shape.

This is worth stating plainly because it is the direct contrast with the sibling family. The sweep's one real defect lives in `recursive_doubling`'s remainder pre-phase, which exists only because the hypercube exchange requires a power-of-two participant set. `reduce_bcast` has no such requirement, so it has no such phase, so it has no such defect. Structural simplicity at awkward sizes is not a minor virtue: agent populations are chosen by how much work there is, not by what factors nicely, and twelve of the twenty-one sizes in this sweep are not powers of two.

The only p-dependent behaviour in the family is the `flat` fallback below $p = 4$ discussed above, which changes the constituents' algorithms at $p = 2$ and $p = 3$ without changing the message count ($2(p-1)$ either way, since flat and binomial both send $p-1$ messages).

6 When to choose this algorithm

6.1 The comparison, with numbers

`reduce_bcast` and `recursive_doubling` are the cleanest algorithm trade-off in the archive, because they are the same collective by two structurally opposite routes and both were measured at all 21 sizes. Logged figures at matching p and campaign:

	messages		rounds		wire tokens		operator applications	
p	red. + bc.	rec. dbl.	red. + bc.	rec. dbl.	red. + bc.	rec. dbl.	red. + bc.	rec. dbl.
8	14	24	6	3	147	384	7	24
10	18	30	8	4	189	450	9	28
16	30	64	8	4	315	1024	15	64
32	62	160	10	5	651	2560	31	160

The messages, rounds and wire tokens are logged. The application counts are read from the implementations: `reduce_bcast` applies the operator once per edge of the binomial tree, $p-1$ times; recursive doubling applies it once per received fold message, which is $\mathrm{pof2}\log_2\mathrm{pof2} + \mathrm{rem}$.

So recursive doubling buys a halving of the round count for 2.6× the messages, 3.9× the wire volume and 5.2× the operator applications at $p = 32$. In MPI that would still be a defensible trade, because α dominates and a message is a message.

6.2 Why the halved round count buys almost nothing here

The AgentMPI setting changes the arithmetic, and this is the substantive finding of the pair.

`reduce_bcast`'s $2\lceil\log_2 p\rceil$ rounds are not $2\lceil\log_2 p\rceil$ equivalent rounds. Half of them are reduce rounds, each of which applies the operator; the other half are broadcast rounds, which apply nothing. The `bcast` docstring explains why they cost so little: every payload is content-addressed, and forwarding moves the *handle*, so an interior rank relays without paraphrasing and without invoking an agent. This is measurable in the trace — at $p = 10$ the nine reduce sends carry 20 tokens each (the envelope `{"acc", "depth", "weight", "vr"}`) and the nine broadcast sends carry one token each, the bare result.

Priced with the repository's own defaults ($\alpha = 12\,\mathrm{s}$, $\beta^{-1} = 45$ tokens/s, `fabric_s` = 3 ms, `beta_in` = 1/8000 s per token), a fold round on a 1200-token payload costs about 38.7 s and a forward round about 0.153 s. `cost.predict` accordingly gives 116.9 s for `reduce_bcast` against 116.5 s for recursive

doubling at $p = 8$, and 194.9 s against 194.1 s at $p = 32$. The latency-optimal algorithm is ahead by four parts in a thousand. `best_algorithm` with `objective="time"` returns `recursive_doubling` at every p on that margin, and with `objective="price"` returns `reduce_bcast` at every p. Given that the time difference is inside any plausible noise band and the price difference is 70%, `reduce_bcast` is the answer, and the code's default is right.

One caveat on the price figure, because it understates rather than overstates the case. `cost.predict` charges the operator-output term as $(p-1) \cdot$ `op_cost_tokens` for *every* algorithm. That is exact for the reduce family, where every algorithm performs $p - 1$ applications, and it is exact for `reduce_bcast`. It is not exact for recursive doubling, which performs 160 applications at $p = 32$; the model charges 31. So the 70% price gap the model reports is a floor, and a corrected model would put the gap nearer a factor of three. The discrepancy is internal to `cost.py` — the `Prediction` object already carries `messages: 160` beside the price computed from 31 — so it needs no measurement to confirm, and it is worth checking against the reduce family, where the assumption holds.

6.3 The property that is not on the cost axes

The reason to prefer this algorithm is not primarily arithmetic. `reduce_bcast` produces *one* result. The operator is applied $p - 1$ times along a fixed tree with the lower virtual rank always the accumulator, so the fold order is canonical and there is exactly one value; `bcast` then delivers that value by handle. The trace proves the byte-identity rather than asserting it: at $p = 32$ all 31 broadcast sends carry the digest `e29c9c180c62`, and at $p = 10$ all 9 carry `4a44dc153642`. One content-addressed object, forwarded, unmodified.

Recursive doubling has each rank compute its own copy. For an exact associative operator the copies are equal and the distinction is void — and this archive confirms as much, since the result digest recursive doubling distributes matches `reduce_bcast`'s at all eight non-power-of-two sizes where both ran. For a lossy operator they need not be equal. AgentMPI takes the trap seriously enough to canonicalise the operand order inside recursive doubling (`if comm.rank < partner or op.commutative`), which removes fold *order* as a source of divergence; what it cannot remove is that the operator is executed independently 160 times instead of 31, and a language model given identical operands twice need not answer twice the same. That is the residual risk, and it is a nondeterminism argument rather than an associativity one.

It is also unmeasured. Across all 496 `coll.*` records in the archive that carry the field, `divergence_risk` is `false` every time. The flag is `op.lossy`, and the operators run under `reduce_bcast` in the sweep are `SUM` (206 records), `UNION` (68) and `GLOSSARY_MERGE` (8) — the first two declared `Associativity.EXACT`, and the third, the only semantic one, appearing *only* under `reduce_bcast`. So the design's semantic-operator × recursive-doubling cell is empty, and no trace in this repository shows a population disagreeing. The argument for `reduce_bcast`'s single canonical result is a guarantee by construction; the argument against recursive doubling's p results is a prediction that has not been tested.

7 Threats to this reading

These runs contain no agents, so the axis on which this algorithm is chosen is the one the sweep does not measure. Every member records `executors: {none: p}`, zero agent calls, zero tokens in and out, $0.0000; `total_busy_s` is 0 and `idle_fraction` is 1.0 at every size,

because `bench_collectives` calls `comm.allreduce(1, SUM)` and returns. All of §3.2 and §3.3 rests on `cost.py`'s parameters and on reading the implementations, not on these traces. What the sweep establishes is structure — $2(p-1)$ messages in two trees at every size, and a nested attribution that adds up — and that is worth having precisely because it is the part a simulation cannot supply.

The measured wall times must not be read as latency, and for this family they invert the conclusion. `reduce_bcast` is *faster* than recursive doubling in the logs at the larger sizes: 0.186 s against 0.228 s at $p = 32$, 0.098 s against 0.128 s at $p = 16$ (`mb-free`). With α effectively zero, cost is dominated by per-message fabric work and by the receive-side poll backoff (`POLL_INTERVAL` 10 ms growing by 1.4× to `MAX_POLL_INTERVAL` 250 ms in `comm.py`), so the algorithm with fewer messages wins and the round count barely registers. That is the opposite regime from the one the algorithm choice is about. Neither the measured ordering nor the modelled ordering should be quoted without saying which regime it belongs to.

The 20-token and 1-token message sizes are framing, not content. The benchmark's payload is the integer 1, so the reduce message is almost entirely its `{"acc", "depth", "weight", "vr"}` envelope and the broadcast message is a single token. The observation that broadcast messages are cheap is therefore *understated* here in one sense and misleading in another: with a real 1200-token artefact the broadcast messages would carry 1200 tokens of volume, and what stays free is not the volume but the operator invocation. I have been careful above to make the latency claim about applications rather than about tokens, and the closed form's volume term, $2(p-1)n$, is effectively untested by a sweep in which $n = 1$.

The nested attribution is a hard-coded table, and it is right only because someone kept it in step with the code. `COMPOSED_ALGORITHMS` lists three entries; if a fourth composed algorithm were added to `algorithms.py` without a matching entry, its family would report a phantom disagreement at every size — which is exactly what happened to this family under the earlier version of the tool. So the ✓ at 21 of 21 rows here is conditional on a piece of duplicated knowledge, and a reader should treat it as a check on the runtime given a correct tool, not as an independent verification of both.

The fold-depth argument is about a lossy operator and every measured run used an exact one. The claim that only the maximum `fold_depth` is meaningful is a reading of what the field is computed from, and it is correct as such. But the reason it matters — that $F(d) = (1-\delta)^d$ is the fidelity of the value a rank holds — depends on a δ that `CostParams` sets to 0.05 by default and that no run in this family measures. `benchmarks/bench_reduce.py` and `bench_fidelity` are where that number would come from; nothing here contributes to it.